

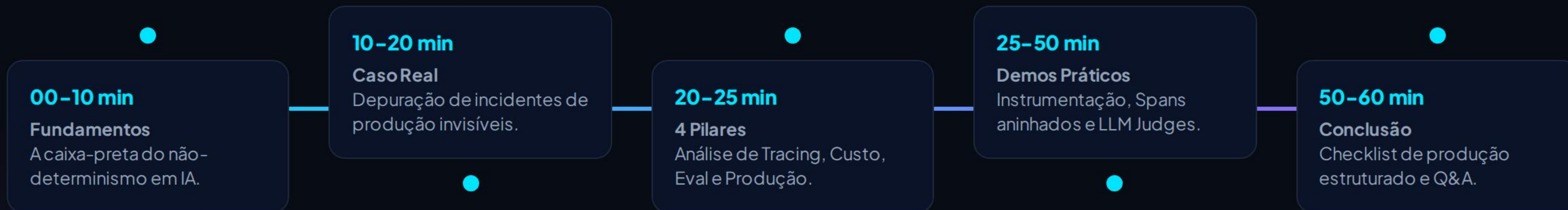
MLFLOW GENAI FRAMEWORK

Observabilidade em IA

Workshop Prático de Engenharia — Validação, Julgamento e Monitoramento de LLMs em Produção

Referência de Projeto: **workshop-observabilidade-ia.pptx** | Módulo: ia-observability

| Roteiro do **Workshop** (1 Hora)



| A Mudança Fundamental: **Software vs. IA**

</> **Software Tradicional**

- **Determinístico:** Input idêntico retorna exatamente o mesmo output.
- **Depuração direta:** Falhas acionam erros explícitos e rastreáveis na stack.
- **Custos previsíveis:** Alocação de recursos fixos e escalabilidade direta.
- **Lógica estática:** Fácil validação via testes unitários e de integração locais.

🧠 **Engenharia de LLM**

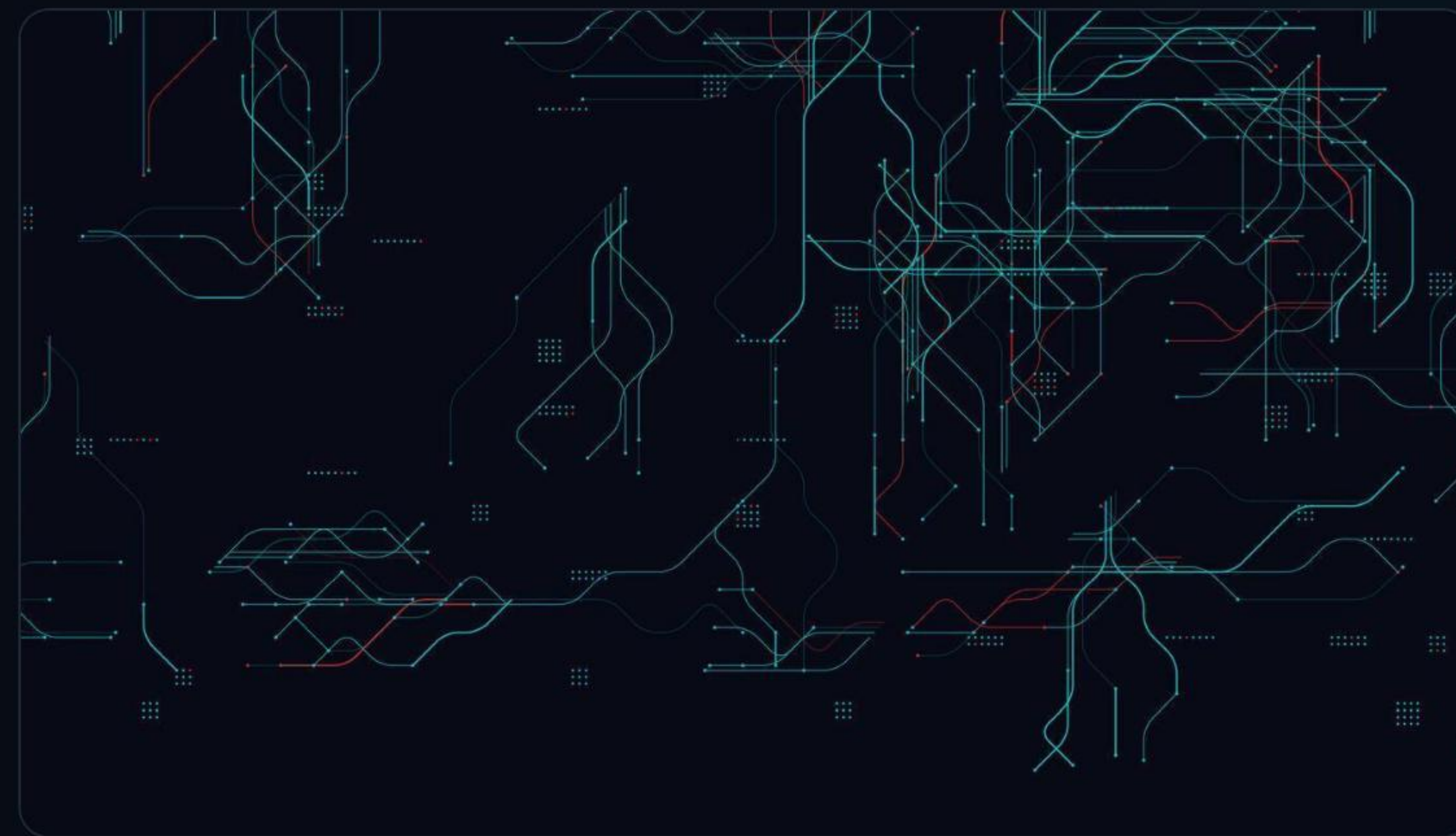
- **Não-determinístico:** Respostas mudam mesmo com o mesmo prompt de entrada.
- **Falhas silenciosas:** Alucinações graves sem acionar erros de código (HTTP 200).
- **Custo dinâmico:** Volume e multiplicação de tokens voláteis por chamada.
- **Pipelines complexos:** Fluxos de RAG + tool calls múltiplos e encadeados.

| O que é Observabilidade em IA?

"A capacidade de entender o estado interno de uma aplicação de IA a partir de suas saídas externas expostas."

Ao rastrear spans, monitorar a contagem de tokens, registrar a latência granular e mapear sistematicamente a qualidade semântica da resposta:

Mensagem Principal: Sem observabilidade, calibrar sua IA é puro chute. Com observabilidade, torna-se **engenharia**.



Mapeamento de Produção: **Dúvida vs. Solução**

Pergunta Crítica da Equipe de Engenharia	Abordagem de Resolução Técnica	Atributo Observado no MLflow
O que o usuário enviou e qual foi a resposta final exata gerada?	Tracing Automatizado	Input / Output payload final
Qual ferramenta (tool) o agente acionou? Houve lentidão ou erro?	Spans Customizados de Tool	Duração interna e payload do Span
Por que o custo de computação explodiu sem aviso no mês?	Token tracking de custos	Metadados e tags de uso de tokens
A resposta gerada estava tecnicamente certa e livre de alucinações?	Avaliação contínua (Judges)	Technical accuracy & Code scores
Qual foi a versão do prompt ativo que gerou o incidente relatado?	Versionamento de prompts	Tag de ID de prompt no trace

Demo 1: Auto-Tracing em 1 Linha

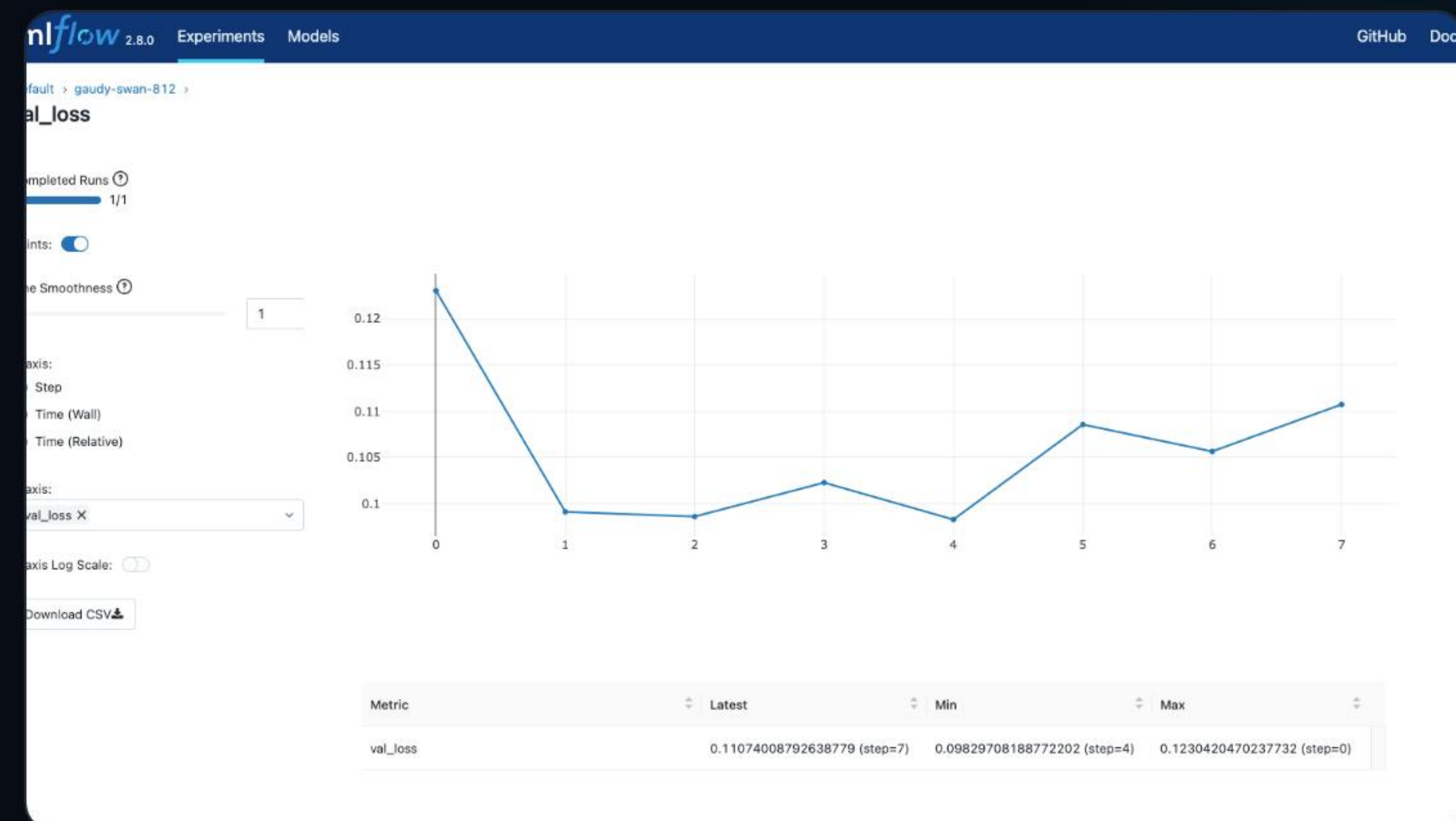
Instrumentação nativa do SDK:

```
PYTHON ENGINE

import mlflow
from ia_observability.config import MODEL_NAME, get_client

@mlflow.openai.autolog() # Captura inputs, outputs, tokens e
latência
def demo_auto_tracing():
    client = get_client()
    response = client.chat.completions.create(
        model=MODEL_NAME,
        messages=[{"role": "user", "content": "O que é
observabilidade?"}]
    )
    mlflow.flush_trace_async_logging() # Sincroniza envio de
logs
```

💡 **Visibilidade imediata:** Traces, latências, custos e dados brutos gerados sem mudar a estrutura de chamadas da aplicação.



| Demo 2: Spans Aninhados (Padrão RAG)

Estruturação de hierarquia de execução:

PYTHON ENGINE

```
@mlflow.trace
def demo_rag_pipeline(question: str) → str:
    context = retrieve_context(question) # Span filho 1
    answer = generate_answer(question, context) # Span filho 2
    return answer

@mlflow.trace(span_type="RETRIEVER")
def retrieve_context(question: str) → str:
    return "Contexto recuperado do Vector Database"

@mlflow.trace(span_type="LLM")
def generate_answer(question: str, context: str) → str:
    return "Resposta final construída"
```

Isolamento de Falhas

Quando a resposta final para o cliente final estiver errada, a visualização de árvore responde instantaneamente:

-  **Falha no Retriever:** Os documentos recuperados não continham a resposta (contexto irrelevante).
-  **Falha na Geração:** O contexto estava correto, mas o modelo de linguagem falhou na lógica de extração.

| Demo 3: Atribuição Manual de Custo

Necessário para modelos customizados e Self-Hosted:

PYTHON ENGINE

```
def _set_usage_and_cost(span, usage) → None:
    input_tokens = usage.prompt_tokens or 0
    output_tokens = usage.completion_tokens or 0

    input_cost = input_tokens * CUSTOM_INPUT_COST
    output_cost = output_tokens * CUSTOM_OUTPUT_COST

    span.set_attribute("mlflow.chat.tokenUsage", {
        "input_tokens": input_tokens,
        "output_tokens": output_tokens,
    })
    span.set_attribute("mlflow.llm.cost", {
        "input_cost": input_cost,
        "output_cost": output_cost,
        "total_cost": input_cost + output_cost
    })
```

Controle Orçamentário

Enquanto provedores como OpenAI e Anthropic têm custos mapeados de forma automática no MLflow, servidores locais ou infraestrutura própria necessitam de injeção explícita de custos.

Utilize esses atributos para rastrear o "Cost Breakdown" e o consumo detalhado por trace de forma a auditar saltos de gastos.

| Demo 4: LLM Judges vs. Code Scorers

LLM Judge (Semântico)

Utiliza modelos de linguagem inteligentes para validar regras subjetivas segundo linguagem natural:

```
Guidelines(  
    name="technical_accuracy",  
    guidelines=(  
        "A resposta deve ser precisa sobre "  
        "MLflow. Nao invente dados."  
    ),  
    model=JUDGE_MODEL,  
)
```

PYTHON ENGINE

Code-Based Scorer (Lógico)

Validação determinística pura escrita diretamente em Python, rodando instantaneamente com custo zero:

```
@scorer  
def no_hallucination_keywords(inputs, outputs):  
    red_flags = ["comprovado cientificamente"]  
    found = [f for f in red_flags if f in str(outputs)]  
    if found:  
        return Feedback(value=False, rationale=found)  
    return Feedback(value=True)
```

PYTHON ENGINE

| Demo 5: `autolog()` vs. Spans Manuais

Aspecto Arquitetural	<code>mlflow.langchain.autolog()</code>	Manual Spans (Padrão Produção)
Suporte a Streaming	Incompatível (Força chamadas de invoke bloqueantes)	Totalmente compatível com geradores de token (.astream)
Gestão de Sessão	Garantido implicitamente por checkpointing genérico	Construído manualmente via Cache / Dicionário de histórico
Controle Fino	Spans criados de forma genérica e automatizada	Inputs e outputs mapeados explicitamente por escopo
Mensagens de Progresso	Retorna logs e metadados apenas no resultado final	Envia logs em tempo real por meio de <code>get_stream_writer()</code>
Associação de <code>trace_id</code>	Requer busca assíncrona pós-chamada no MLflow	Capturado instantaneamente na inicialização do ciclo de span

| Demo 6: Amostragem e Feedback Ativo

1. Amostragem Seletiva (Sampling Ratio):

```
@mlflow.trace(sampling_ratio_override=1.0)
def critical_payment():
    # Processamento crítico de checkout (100%)
    return ...

@mlflow.trace(sampling_ratio_override=0.1)
def general_chat():
    # Amostragem padrão de logs rotineiros (10%)
    return ...
```

PYTHON ENGINE

2. Registro de Feedback Humano nos Traces:

```
trace_id = mlflow.get_last_active_trace_id()
mlflow.log_feedback(
    trace_id=trace_id,
    name="user_rating",
    value=False, # 👍 ou 👎
    rationale="Ignorou minha pergunta sobre preço",
    source=AssessmentSource(source_type="HUMAN")
)
mlflow.flush_trace_async_logging() # Garante escrita
```

PYTHON ENGINE

| Checklist de Implantação em Produção

- ✅ **Habilite o auto-tracing básico como fundação:** Ative `mlflow.openai.autolog()` ou o equivalente para LangChain/LlamaIndex durante as etapas de prototipagem rápida de prompts.
- 🔧 **Isole fluxos complexos com spans aninhados:** Decore funções de banco vetorial e APIs externas de forma a identificar exatamente se a lentidão decorre do modelo de linguagem ou do banco.
- ⚖️ **Una estratégias de avaliação:** Escreva scorers determinísticos em código nativo para regras absolutas e utilize LLM Judges seletivamente para mapear a qualidade subjetiva das conversas.
- ☁️ **Configure amostragem e feedback ativo em produção:** Colete avaliações (👍/👎) diretamente do front-end vinculadas ao `trace_id` correspondente, diminuindo os dados armazenados em rotas frias.

| Image Sources



https://static.vecteezy.com/system/resources/previews/068/364/830/non_2x/circuit-board-background-abstract-neural-grid-with-glowing-lines-and-data-paths-i-deal-for-future-technology-ai-interfaces-and-cyber-projects-illustration-vector.jpg

Source: www.vecteezy.com



<https://mlflow.org/docs/latest/assets/images/tracking-metrics-ui-temp-ffc0da57b388076730e20207dbd7f9c4.png>

Source: mlflow.org